Design principles, patterns, and backend architecture

Companion guide for Sreedhar Suswaram | Senior Software Engineer interview | October 22, 2026

This guide builds on your technical and coding preparation. It uses commercial payments, reconciliation, identity, and user-management examples because those appear in your resumes. The examples are **practice designs**, not descriptions of American Express systems. State what you actually implemented when discussing your own work.

1. How to study this guide

Design principle = a rule for making choices, such as keeping responsibilities clear. **Design pattern** = a reusable way to organize classes or objects for a recurring problem. **Architecture pattern** = a way to organize larger components and their communication. An implementation may use several patterns, but every pattern should solve a named problem.

In the upcoming technical round, prepare to explain a design choice while coding or discussing a project. For the later system-design round, practice drawing boundaries, data flow, failure paths, and tradeoffs. Do not spend this week memorizing a pattern catalog. Learn the problem, the mechanism, the cost, and when to avoid the pattern.

Fit into the existing 20-hour plan

When	Time	Focus	Checkpoint
During Java fundamentals	35 min	SOLID, composition, immutability, Strategy, Factory, Builder	Refactor one conditional-heavy example aloud
During Spring and API study	35 min	Layering, Adapter, Repository, dependency inversion, transactions	Trace a request and name each responsibility
During project-story practice	30 min	Connect one real decision to its tradeoff and test	Give a two-minute design story with personal ownership
During production review	35 min	Outbox, retry, circuit breaker, bulkhead, idempotency	Explain what happens after a timeout or duplicate event
During a mock follow-up	15 min	Pattern selection and critique	Answer without relying on buzzwords

That is **2 hours 30 minutes** inside your 20-hour plan. For the later system-design round, revisit the architecture section more deeply. If the interviewer asks about a technology you used, give its actual implementation details before naming a pattern.

Use this answer structure: **"The problem was __. I chose __ because __. The cost was __. I handled failure by __. We verified it with __."** Speaking the tradeoff is what makes an answer sound senior.

2. Design principles you should know

1. Single responsibility principle (SRP) [High]

Plain English: A component should have one coherent reason to change. This does not mean it must have only one method. A payment class that validates requests, calculates fees, saves records, and sends email may mix unrelated changes.

Professional answer: "I group behavior around one responsibility and keep boundaries clear. For example, payment policy, persistence, and notification change for different reasons, so I separate them when that separation makes changes and tests safer."

When to use and cost: Split when changes repeatedly touch unrelated logic or tests become difficult. Too many tiny classes can make a simple flow hard to follow. Ask: "What business decision belongs here, and who owns it?"

2. Open/closed principle (OCP) [High]

Plain English: Add a new variation with limited changes to stable, existing code. If every new payment method forces edits to one large switch, the design may need an extension point.

Professional answer: "I use an extension point when variations are real and growing. A PaymentMethod interface can let a new method be added without repeatedly changing the orchestration. I still avoid abstractions for speculative future features."

Trap: A change in business policy may legitimately change existing code. OCP is a design direction, not a ban on editing a class.

3. Liskov substitution principle (LSP) [High]

Plain English: If B is used wherever A is expected, B should honor A's promises. A subclass that throws for a supported parent operation or silently changes its meaning breaks that promise.

Professional answer: "I design subtypes around behavioral contracts, not only matching method signatures. A replacement should preserve the expectations callers depend on, including error behavior and business invariants."

Example: A read-only account should not extend a mutable account contract with `withdraw()` that always throws. Separate capabilities or use composition. Test the same contract against each implementation.

4. Interface segregation principle (ISP) [Medium]

Plain English: A caller should depend on only the operations it needs. One giant financial-platform interface makes unrelated clients know too much.

Professional answer: "I prefer focused interfaces based on consumers. A reconciliation reader should not need payment-submission or user-administration methods. This makes dependencies and permissions easier to reason about."

Cost: Too many one-method interfaces can create noise. Split where consumers genuinely need different contracts.

5. Dependency inversion principle (DIP) [High]

Plain English: Business logic should depend on an appropriate contract rather than on a specific database, HTTP client, or vendor SDK. An adapter connects the contract to the outside system.

Professional answer: "I keep core business policy independent of infrastructure details. For example, a reconciliation service can depend on a `TransactionStore` contract while an adapter implements it with DB2 or another store. This allows focused tests and controlled replacement."

Clarify: Dependency injection supplies an implementation. Dependency inversion is the design of which component owns and depends on the interface. Constructor injection does not automatically make a design good.

6. Cohesion and coupling [High]

Plain English: Cohesion means related work stays together. Coupling means components rely on each other's details. Aim for cohesive components with narrow, understandable connections.

Professional answer: "I evaluate a boundary by asking whether its logic belongs together and how many implementation details leak across it. Clear contracts and data ownership reduce the number of changes needed for a new feature."

Example: If changing a database column requires edits in five unrelated services, a storage detail has leaked through boundaries. Low coupling does not mean no collaboration: services still need contracts.

7. Composition over inheritance [High]

Plain English: Composition builds behavior using collaborating objects. Inheritance models a true is-a relationship. Composition often makes policy selection easier without a deep class tree.

Professional answer: "I use composition for variable behavior, such as fee calculation or provider integration. I reserve inheritance for a stable substitutable relationship. This keeps each policy independently testable."

Example: `PaymentProcessor` uses `FeePolicy`. It does not need subclasses for every market, channel, and currency combination. Those axes can combine without multiplying subclasses.

8. DRY, KISS, and YAGNI [High]

Plain English: DRY avoids repeating the same knowledge in many places. KISS favors the simplest design that meets the requirement. YAGNI says do not build a feature solely because it may be useful someday.

Professional answer: "I remove duplicated business rules, but I do not extract unrelated code just because two blocks look similar. I start with the simplest design that meets current behavior and add extension points when variation becomes real."

Trap: Two similar-looking transformations may follow different business policies. Forcing them into one shared helper can create many flags and make changes riskier. Duplicate syntax and duplicate knowledge are not always the same thing.

9. Encapsulation and invariants [High]

Plain English: Encapsulation hides implementation details and protects rules that must always hold. An invariant is such a rule: an approved payment cannot have a negative amount, or an account change must be authorized.

Professional answer: "I keep state changes behind operations that enforce the business rules. I do not expose mutable internals and hope every caller validates them correctly. I also use database constraints for rules that must hold across concurrent application instances."

Example: A `Payment` may expose `approve()` with checks rather than a public `setStatus()`. Client-side validation and DTO validation do not replace domain or database protection.

10. Design for testability and observability [High]

Plain English: A useful design makes correct behavior and failure behavior visible. Small contracts make tests easier; logs, metrics, and traces make production behavior explainable.

Professional answer: "I build seams around external dependencies and test business rules independently. I also make important state transitions and failure paths observable so an incident can be diagnosed from evidence rather than guesses."

Example: Test a retry rule with a fake provider, then integration-test real persistence and deduplication constraints. Track request ID, business ID, state, latency, and error category without logging secrets.

11. Fail fast at boundaries, recover deliberately [Medium]

Plain English: Reject invalid input before doing expensive or irreversible work. For failures that may be temporary, recover only when the operation is safe to repeat.

Professional answer: "I validate preconditions early, return a clear error for invalid requests, and reserve retries for known transient failures. For a timed-out payment call, I first determine whether the operation completed before retrying."

Trap: "Fail fast" does not mean crash every service on a recoverable downstream problem. The boundary, recovery contract, and customer impact matter.

12. Separate policy from mechanism [Medium]

Plain English: Policy is the business decision; mechanism is how the system performs it. "Who may approve a payment?" is policy. "How do we read the role from a token?" is mechanism.

Professional answer: "I keep business decisions explicit and avoid burying them in transport or persistence code. A policy can then be reviewed by business owners and tested without making an HTTP call or using a real database."

Example: `ApprovalPolicy.canApprove(principal, payment)` should express the rule. A security adapter obtains the principal, and a repository loads the payment.

3. Object-level design patterns

Patterns are useful vocabulary, not proof of good design. In this section, say **problem -> shape -> example -> cost**. When an interviewer asks "Which pattern would you use?", clarify whether behavior varies, a vendor must be isolated, work is asynchronous, or state changes are constrained.

Strategy [High]

Plain English: Put each interchangeable rule behind the same small contract. Select the rule based on context.

Professional answer: "I use Strategy when one behavior has several valid policies, such as fee calculation by product. The caller uses one contract and each policy is tested separately. I avoid it for a single simple branch."

Shape: `FeePolicy.calculate(payment)`. Implementations could be domestic and international fee policies. Be careful that variation may depend on more than country; make selection rules explicit. **Cost:** More classes and a policy-selection mechanism.

Factory Method or simple factory [High]

Plain English: Put object creation and selection in one place. A simple factory can choose a `PaymentHandler` based on payment type. Factory Method specifically lets subclasses decide which concrete object to create.

Professional answer: "I separate complex creation from the business flow. If selection rules are centralized, a factory can return the correct handler while the processing code works with the handler contract. For a trivial object, direct construction is clearer."

Trap: Do not call every constructor wrapper a Factory Method. An interviewer may ask you to distinguish the formal pattern from a simple factory.

Builder [High]

Plain English: Construct a complex object step by step while making required data and validation clear.

Professional answer: "I use Builder when a request or configuration has many optional fields and construction should be readable. I validate required fields at build time and favor an immutable result."

Example: A payment instruction has mandatory tenant, amount, currency, and beneficiary, plus optional remittance details. **Cost:** Extra code; records or constructors may be enough for a small object.

Adapter [High]

Plain English: Translate an external interface into the interface your application wants.

Professional answer: "I use Adapter to keep provider-specific APIs and data shapes outside business logic. Our core service calls a stable internal contract; the adapter handles provider request mapping, errors, and transport details."

Example: An old file-based payment interface and a new real-time service can implement the same internal `PaymentSubmissionPort` during migration. **Cost:** Translation logic and another layer; do not hide differences that change business semantics.

Facade [High]

Plain English: Give callers one simple entry point to a complicated subsystem. The facade coordinates existing capabilities.

Professional answer: "I use a facade when clients should not orchestrate several low-level components themselves. For example, a user-onboarding facade can coordinate identity creation, role assignment, and audit recording behind one application operation."

Cost: A facade can become a very large service if it absorbs all business rules. Keep each collaborator's responsibility visible. A facade simplifies access; an adapter changes an interface.

Decorator [Medium]

Plain English: Wrap a component with another component that has the same contract and adds behavior.

Professional answer: "Decorator can add cross-cutting behavior such as metrics or caching around a repository or client without changing its core implementation. I use it when that behavior is composable and the wrapper's effect remains clear."

Example: `TimedPaymentGateway` measures a gateway call and delegates to `PaymentGateway`. **Cost:** Several wrappers can obscure call order and failure behavior. A cache wrapper needs a freshness policy.

Proxy [Medium]

Plain English: A proxy stands in front of another object and controls access to it. It may add transactions, security, lazy loading, or remote access.

Professional answer: "I expect proxy behavior at framework boundaries. In Spring, transaction advice commonly runs through a proxy, so a method calling another method on the same instance may bypass that advice. I test transaction behavior at the actual call boundary."

Difference from Decorator: Both wrap, but a proxy often controls access or lifecycle, whereas a decorator intentionally adds composable behavior. The structures can look similar; discuss intent and effects. See Spring transaction implementation.

Observer and publish/subscribe [Medium]

Plain English: One component announces an event and interested components react. In-process Observer means listeners in the same process. A message broker makes communication cross-process, with different durability and delivery concerns.

Professional answer: "I publish an event when other components need to react to a completed state change without being called synchronously. I distinguish a local callback from a durable event and plan for duplicates, ordering, and failed consumers."

Example: User-created event triggers audit or notifications. Do not send a durable event before its source transaction commits unless the design handles rollback.

Command [Medium]

Plain English: Represent an operation as a value. This can separate receiving a request from executing, recording, retrying, or scheduling it.

Professional answer: "A command makes the requested action explicit, such as `ApprovePayment`. A handler validates and executes it. This can help with workflow, auditing, or async dispatch, but I do not add command objects to simple code without a need."

Trap: A command is an instruction or intent; an event states something that already happened. `ApprovePayment` and `PaymentApproved` are not interchangeable.

State [High]

Plain English: Behavior changes depending on an object's state. Representing valid transitions prevents scattered status checks.

Professional answer: "I model payment status as explicit transitions and reject invalid changes. For a complex workflow, a State pattern or transition table can make the rules reviewable. For a small fixed workflow, an enum with a validated transition method may be simpler."

Example: `CREATED -> AUTHORIZED -> SETTLED`, with `FAILED` paths. Define whether reversal after settlement is a new compensating operation rather than simply moving status backward. **Cost:** More structure, especially if every status becomes a class.

Template Method [Medium]

Plain English: A base class fixes the overall algorithm and lets subclasses fill certain steps.

Professional answer: "Template Method fits a stable process with a few controlled variations. For example, each import may share validate, transform, persist, and report steps. I prefer composition if the variations combine in many ways or inheritance makes the hierarchy brittle."

Trap: A subclass must preserve the base process's contract. Hidden calls from the base class to overridable methods can complicate debugging.

Chain of Responsibility [Medium]

Plain English: Pass a request through a sequence of handlers; each handler may process, reject, or pass it on.

Professional answer: "I use a chain when independent checks need an ordered pipeline, such as authentication, input limits, and audit enrichment. I document the order and whether a handler can stop processing."

Cost: Debugging becomes hard if order is implicit or failures disappear in the chain. Spring filters or interceptors are familiar examples, though their exact semantics differ.

Repository and Unit of Work [High]

Plain English: A Repository gives domain-oriented access to stored objects. A Unit of Work tracks related changes that should commit as one unit. In Spring/JPA, a persistence context and transaction often supply much of the Unit of Work behavior.

Professional answer: "I keep persistence queries behind meaningful repository operations and place the transaction around the business operation. I do not assume a repository method makes a multi-step workflow atomic; that depends on the transaction boundary and database guarantees."

Cost: A generic repository with dozens of vague methods can hide inefficient queries. A repository is not automatically an abstraction over every storage engine.

Singleton: recognize it, use sparingly [Medium]

Plain English: A Singleton controls creation so there is one instance within a defined scope. In a distributed service, each process still has its own instance.

Professional answer: "I usually let the application container manage shared, stateless services. I do not use a global singleton for request state or payment deduplication because it cannot coordinate separate instances or survive restarts."

Trap: A Spring singleton bean is scoped to an application context. Shared mutable fields still need concurrency protection.

4. Architecture patterns and when to choose them

1. Layered architecture [High]

Plain English: Group work by role, such as API, application service, business model, and persistence. A request moves through clear responsibilities.

Professional answer: "Layering helps keep transport, business rules, and data access understandable. In a Spring service, the controller handles HTTP, the application service coordinates a use case, and repositories persist state. I watch for business rules leaking into controllers or repositories."

Use when: A service has a straightforward workflow. **Cost:** Too many pass-through layers; an anemic domain model if every rule is pushed into one large service class.

2. Hexagonal architecture / ports and adapters [High]

Plain English: Put core business logic in the center. Define ports for what it needs and adapters for HTTP, databases, queues, or vendor systems. The core should not need to know the vendor's API.

Professional answer: "I use ports and adapters when external integrations or storage may change, or when business rules need isolated tests. The application owns a business-oriented port, and an adapter translates to the technology. I do not add a separate interface and adapter for every trivial class."

Example: `SubmitPaymentUseCase` depends on `PaymentStore` and `ClearingGateway`. REST and batch entry points call the same use case. DB2 and provider adapters implement the ports. **Cost:** More boundary code and mapping. See AWS hexagonal architecture guidance.

3. Modular monolith [High]

Plain English: One deployable application can still have strong internal modules, such as identity, payment, and reconciliation. Each module owns its rules and communicates through clear interfaces.

Professional answer: "I consider a modular monolith when teams need clean boundaries but separate deployments and distributed failure handling are not justified. It is a useful starting point because local transactions and debugging are simpler. I keep module ownership explicit so future extraction remains possible."

Cost: Shared deployment and runtime; boundaries can erode if every module reads another's tables. A module is not isolated merely because it has a package name.

4. Microservices with owned data [High]

Plain English: Independent services own capabilities and, ideally, control their own data. They communicate through APIs or events. This helps independent change but creates network, consistency, testing, and operations costs.

Professional answer: "I choose service boundaries around business capabilities and team ownership, not around small classes. I account for API contracts, data ownership, observability, deployment, and partial failure. I do not split a cohesive workflow simply because microservices are popular."

Caution: Database-per-service means one service controls access to its data; it does not require a separate database server for every service. Cross-service reports need API composition, materialized views, or events. See database-per-service pattern.

5. Synchronous request/response [High]

Plain English: A caller sends a request and waits for an answer. It is simple when the answer is needed now. The caller's response time and availability depend on the downstream path.

Professional answer: "I use a synchronous call when the caller needs an immediate decision. I set a deadline, bound concurrency, and define what a timeout means. I avoid a long chain of dependent calls when it makes latency and failure hard to control."

Example: Validate a payment request and return whether it was accepted for processing. Do not promise final settlement if that occurs later.

6. Event-driven and publish/subscribe architecture [High]

Plain English: A service emits a fact about a completed change; other services respond when they receive it. This reduces direct coupling but introduces delivery delay and duplicate or out-of-order events.

Professional answer: "I use events when downstream consumers can react independently and eventual consistency is acceptable. I define event ownership, schema evolution, idempotent consumers, ordering needs, and how failed events are replayed."

Example: `PaymentAccepted` can trigger audit and status projection. Do not put sensitive data in an event merely for convenience. A command tells one component to do something; an event tells interested components something happened.

7. Transactional outbox [High]

Plain English: Saving business data and publishing an event are two writes. One can succeed while the other fails. An outbox saves the business change and event record in one local database transaction; a relay publishes the event afterward.

Professional answer: "I use an outbox when a committed state change must reliably produce an event. The local transaction writes both business and outbox rows. A relay may publish more than once, so consumers still need idempotency and monitoring."

Cost: Outbox storage, relay, lag, cleanup, ordering decisions. It does not make an external payment provider part of the local transaction. See AWS transactional outbox.

8. Saga [Medium; later design round]

Plain English: A workflow across several services is split into local steps. If a later step fails, the system performs a defined compensating action where possible. Compensation is a new business operation, not a magic database rollback.

Professional answer: "I consider a saga when one business process crosses independently owned services and a single database transaction cannot cover it. I define states, retry and compensation behavior, and what happens after an irreversible step. For a simple workflow within one database, a local transaction is safer and easier."

Example: Reserve funds, submit instruction, record confirmation. If submission fails before an irreversible action, release the reservation. If the external outcome is unknown, reconcile before resubmitting. **Cost:** State tracking, delayed consistency, failed compensation and operational complexity. See Microsoft saga pattern.

9. CQRS [Medium; later design round]

Plain English: Commands change state; queries read it. CQRS separates their models so each can fit its job. This can be as small as separate code paths in one database or as large as a separate read store.

Professional answer: "I use CQRS only when read and write needs differ enough to justify separate models. For example, payment changes may enforce strict invariants while reconciliation dashboards need a denormalized view. I would define how the view is updated and what users see during lag."

Cost: More code and consistency management. CQRS does not require event sourcing, a message broker, or microservices. See Microsoft CQRS pattern.

10. Event sourcing [Low for this round]

Plain English: Store a sequence of events as the source of truth and rebuild current state from them. An audit log beside a normal database is not automatically event sourcing.

Professional answer: "Event sourcing can provide a detailed history and support rebuilding projections, but it adds schema evolution, replay, ordering, and operational complexity. I would use it only for a clear requirement that outweighs those costs."

Example: Account ledger events may be valuable, but financial audit requirements alone do not force full event sourcing. Know snapshots and replay only at a conceptual level now.

11. Cache-aside [High]

Plain English: Read the cache first. On a miss, read the database and fill the cache. When source data changes, invalidate or update the cached copy according to a freshness policy.

Professional answer: "I use cache-aside for repeated reads where stale data is acceptable within a defined bound. I set tenant-aware keys, TTLs, invalidation rules, and stampede protection, then measure hit rate and tail latency. I keep authoritative payment state and permissions on a deliberate consistency path."

Cost: Staleness, extra failure mode, memory, invalidation races. A 35% latency improvement is useful only when the baseline and workload are known. See Redis cache-aside guidance.

12. Circuit breaker, retry, timeout, and bulkhead [High]

Plain English: A timeout limits waiting. A retry tries again when failure may be temporary and repetition is safe. A circuit breaker stops calls to a dependency that keeps failing. A bulkhead limits how much of the application's resources that dependency can consume.

Professional answer: "I combine bounded timeouts, selective retries, a circuit breaker, and separate resource limits according to the dependency's failure behavior. I also define a safe fallback or clear error. For payments, a timeout is an uncertain outcome, so idempotency and reconciliation must be part of retry design."

Cost: A retry storm can make an outage worse; a circuit breaker can reject a recovered service until probing succeeds; bulkheads use extra capacity. Tune from latency and error data. See Microsoft circuit breaker and bulkhead.

13. Queue-based load leveling [Medium]

Plain English: Put work into a durable queue when producers can send faster than consumers can safely process. Consumers take work at a controlled rate.

Professional answer: "I use a queue when the caller does not require immediate completion and burst traffic would overload a dependency. I define capacity, acceptance response, retry, duplicate handling, dead-letter treatment, and backlog monitoring."

Cost: Delayed results, eventual consistency, operational queues, ordering limitations. A queue does not remove a bottleneck; it changes how you absorb and expose it. See Microsoft load-leveling pattern.

14. Strangler Fig migration [Medium]

Plain English: Replace a legacy system a piece at a time. Route selected capabilities through new code while the old system continues handling the rest.

Professional answer: "For a risky modernization, I favor a controlled migration by capability or customer segment. I define routing, data ownership, reconciliation between old and new behavior, rollback, and a retirement plan for the legacy path."

Example: Move one payment type or client group from file-based processing to a new service, compare results, expand traffic gradually, then remove the old path. **Cost:** Temporary duplication and complex coexistence. See Martin Fowler's Strangler Fig pattern.

5. Worked design: submit a commercial payment

This is a **practice scenario**, not a claim about your employer's implementation. The interviewer asks: "Design the backend for submitting a commercial payment instruction. A customer might retry after a timeout. Downstream processing is asynchronous."

Step 1: Define the contract

Ask: What is the final business outcome? Is the API acknowledging receipt or settlement? How are tenant, amount, currency, beneficiary and authorization validated? What throughput and response-time goal matters? How long should duplicate keys be remembered? Which downstream outcome can be uncertain?

Proposed contract: `POST /payment-instructions` accepts a tenant-scoped idempotency key and a validated request. A successful response means **accepted for processing**, not settled. Return a durable instruction ID and a status resource. A repeated same-key/same-payload request returns the recorded outcome; same key with a different payload is a conflict.

Step 2: Draw the main flow

```
Client
  -> API: authenticate; authorize tenant and account; validate request
  -> Application service: check idempotency and business rules
  -> Local database transaction: instruction + key/result + outbox event
  -> Response: accepted instruction ID and current status

Outbox relay -> durable broker -> payment processor -> provider
                                        |                  |
                                        +-> status update <-+
Status API -> authoritative instruction state
```

The core service depends on a payment store and an external gateway **port**. The REST controller, database repository, outbox relay, and provider client are **adapters**. The domain model protects status transitions. A local transaction protects instruction, deduplication record and outbox record. A separate worker handles the external call.

Step 3: Walk through failures

Failure	Expected behavior	Check to make
Client retries after response was lost	Same key and payload find the existing instruction	No second instruction or charge
Two identical requests arrive together	Durable uniqueness makes one the winner	Concurrent test across instances or transactions
Database commit fails	No accepted instruction or outbox event	Rollback and error response
Relay publishes twice	Consumer recognizes duplicate event	One business effect; observable duplicate count
Provider times out	Outcome remains uncertain until queried or reconciled	Do not blindly submit again
Downstream backlog grows	Limit work and expose pending status; alert on age	Queue age, depth, error rate
Permission is revoked	Each new operation checks current authorization policy	Cross-tenant and revoked-role tests

Step 4: Explain tradeoffs

Synchronous provider submission could give faster final feedback, but lengthens request latency and couples availability to the provider. Asynchronous submission gives a quick durable acceptance, but customers need a status view and understand pending outcomes. A local database transaction is enough for local state and the outbox; it cannot atomically include the remote provider. The design needs provider-side idempotency or reconciliation for uncertain outcomes.

Polished 90-second answer: "I would first separate accepting the instruction from final processing. The API authenticates the caller, verifies tenant and account access, and validates the request. I would require a tenant-scoped idempotency key and store the instruction, deduplication result, and outbox event in one local transaction. The response would say 'accepted' with an instruction ID. A worker publishes and processes the event, with duplicate-safe handling and monitored retries. The hardest case is a provider timeout: I would treat that as an uncertain outcome, check the provider using a stable reference or reconcile it, and avoid a blind second charge. I would measure pending age, error rate, duplicate attempts, and final outcomes. This adds asynchronous workflow complexity, but it isolates the API from provider latency and gives a clear recovery path."

Step 5: Tie back to your real experience

Your resumes discuss payment modernization, Java APIs, identity, caching, reliability, and reconciliation. Choose **one** real project and fill in: Which parts of this practice design did you personally build? Which parts were already present? Which failure actually occurred? Which metric can you verify? Do not present this example system as one you implemented unless it matches your work.

6. Fast comparisons interviewers ask

Choice	First option	Second option	Decision question
Strategy vs State	Selects a policy	Models state-dependent behavior and transitions	Does behavior vary by selected rule or by lifecycle state?
Adapter vs Facade	Translates an incompatible interface	Simplifies access to a complex subsystem	Are you changing the interface or hiding orchestration?
Decorator vs Proxy	Adds composable behavior	Controls access or lifecycle	What is the wrapper's intent and effect?
Factory vs Builder	Selects or creates an implementation	Constructs a complex value step by step	Is the problem which object to create or how to assemble one?
Layered vs Hexagonal	Separates code by responsibility	Makes core policy depend on ports rather than infrastructure	Are external dependencies changing or isolated tests important?
Modular monolith vs microservices	One deployment with modules	Independent deployments and owned data	Do team autonomy and scaling needs justify distributed costs?
Event vs command	Describes a completed fact	Requests an action	Has the change happened yet?
Local transaction vs saga	Atomic within one database boundary	Coordinates local steps across services	How many owners and data stores must change?
CQRS vs event sourcing	Separates reads and writes	Stores events as the source of truth	Are you optimizing read/write models or preserving all changes for replay?
Retry vs circuit breaker	Repeats a safe transient operation	Temporarily stops calls likely to fail	Is recovery likely soon, or is persistent failure harming the system?
Bulkhead vs rate limit	Isolates internal resource pools	Restricts incoming usage	Where must overload be contained?
Cache vs read model	Copies data for faster access	Builds a purpose-specific projection	Is the need speed for the same value or a different query shape?

7. Practice questions and model answers

Question 1: "Explain SOLID without reciting the letters."

Answer: "SOLID helps me keep responsibilities clear, define reliable contracts, and control dependencies. For example, if I add a new payment route, I would not want to modify one giant class that handles validation, fees, storage, and provider calls. I would separate stable orchestration from the parts that vary. I use those principles when they make changes and tests safer, rather than creating abstractions everywhere."

Question 2: "Why would you choose Strategy over a switch statement?"

Answer: "If there are only two simple fixed cases, a switch can be clearer. I move to Strategy when policies grow, change independently, or need separate tests. Then each policy implements one contract and the caller does not contain every rule. I would also make the selection rule explicit so it does not become a hidden second switch."

Question 3: "How is Hexagonal different from layering?"

Answer: "Both can separate responsibilities. Hexagonal makes the dependency direction more explicit: business code defines the ports it needs, and database or API adapters implement them. That is useful when integrations change or business logic needs isolated tests. Layering alone does not guarantee that the business layer is free from infrastructure details."

Question 4: "When would you choose a modular monolith over microservices?"

Answer: "I would start with strong modules if the workflow is cohesive and one team can deploy it safely. A modular monolith keeps local transactions and debugging simpler. I would split services when independent ownership, release cadence, or scaling has enough value to justify network calls, data consistency, monitoring, and operational overhead."

Question 5: "Does the outbox guarantee exactly-once processing?"

Answer: "No. It protects the local database change and event record from a dual-write gap. The relay may publish a message more than once, and a consumer may retry. I would make the consumer idempotent using a stable event ID or business key and check the effect in durable storage."

Question 6: "Can you roll back a saga?"

Answer: "A saga uses compensating business actions after local transactions have committed. It is not a single database rollback. Some effects, such as an external payment already settled, may be irreversible; the right response could be a reversal or manual reconciliation. I would define that behavior before relying on the saga."

Question 7: "What if the provider call times out?"

Answer: "A timeout tells me our response did not arrive within the deadline; it does not tell me whether the provider acted. I would record an uncertain state and query or reconcile using a stable reference. I would retry only under the provider's idempotency contract and the business rule for duplicate prevention."

Question 8: "Which pattern did you use in your last project?"

Answer template: "The problem was [actual problem]. I used [pattern or design] by [specific implementation I owned]. I considered [alternative]. The tradeoff was [real cost], and I verified it with [test or production measure]." If a formal pattern name does not fit, describe the design without forcing a label.

Rapid oral drill

Give each answer in 30-45 seconds, then add one tradeoff: SRP versus tiny classes; DIP versus dependency injection; Strategy versus State; Adapter versus Facade; Repository versus generic CRUD wrapper; modular monolith versus microservices; command versus event; outbox versus saga; CQRS versus event sourcing; retry versus circuit breaker; timeout versus cancellation; bulkhead versus rate limit; cache invalidation; strangler migration rollback; one pattern you personally used.

8. Final one-page recall

- **Name the problem before the pattern.** A pattern is a tool, not the goal.
- **Principles:** cohesive responsibility; reliable contracts; business policy independent from infrastructure; protect invariants.
- **Simple object patterns:** Strategy varies a rule; Factory selects creation; Builder assembles a complex value; Adapter translates; Facade simplifies; State protects lifecycle transitions.
- **Architecture:** layers separate roles; hexagonal protects the core; modular monolith keeps modules in one deployment; microservices add independent ownership and distributed costs.
- **Data:** local transactions protect one boundary; outbox connects a committed change to an event; saga coordinates local steps; CQRS separates read and write models.
- **Failures:** deadlines, retry safety, circuit breakers, bulkheads, idempotency, reconciliation, and monitoring must fit together.
- **Professional answer:** problem -> choice -> implementation -> tradeoff -> failure handling -> evidence.
- **Honesty:** "I implemented __; my team used __; I understand __ conceptually." These are different claims.

Two questions for your interviewer: "How does the team decide service boundaries and data ownership?" and "Which failure mode has been most challenging in your payment or servicing workflows?"

Reference scope: Official documentation linked near the relevant architecture patterns was consulted on October 22, 2026. These patterns are general practice, not a claim about an American Express internal design. The interview examples are tailored to the experience described in the four resume versions you supplied.